

Struct, Operator Overloading & Classes

Unit-04

Struct

Unit-04.01

In C++, a **struct** is a **user-defined aggregate data type** that can contain **member variables (fields)** and **member functions (methods)**. It is very similar to a **class**, with the main difference being that members of a **struct** are by default public, while members of a **class** are by default private.

Struct.....

```
#include <iostream>
using namespace std;
// Define a struct named 'Person'
struct Person {
    // Member variables
    string name;
    int age;

    // Constructor
    Person(const string& n, int a) : name(n), age(a) {}

    // Member function
    void printInfo() {
        cout << "Name: " << name
        << ", Age: " << age << endl;
    }
};
```

```
18  int main(){
19      Person person("Keshav", 999);
20      person.printInfo();
21  }
```

> ./a.out

Name: Keshav, Age: 999

Struct [Constructor].....

```
#include <iostream>
using namespace std;
// Define a struct named 'Person'
struct Person {
    // Member variables
    string name;
    int age;
    // Constructor
    Person(const string& n, int a) : name(n), age(a) {}

    // Member function
    void printInfo() {
        cout << "Name: " << name
        << ", Age: " << age << endl;
    }
};
```

Let's do some dissection.....

```
18 int main(){
19     Person person("Keshav", 999);
20     person.printInfo();
21 }
```

```
> ./a.out
Name: Keshav, Age: 999
```

Struct [Constructor].....

Let's do some dissection.....

```
#include <iostream>
using namespace std;
// Define a struct named 'Person'
struct Person {
    // Member variables
    string name;
    int age;

    // Member function
    void printInfo() {
        cout << "Name: " << name
        << ", Age: " << age << endl;
    }
};
```

DELETE CONSTRUCTOR??

```
18 int main(){
19     Person person("Keshav", 999);
20     person.printInfo();
21 }
```

```
> g++ basics001.cpp
basics001.cpp:17:12: error: no matching constructor for initialization of 'Person'
    Person person("Keshav", 999);
           ^~~~~~
```

Struct [Constructor].....

Let's do some dissection.....

```
#include <iostream>
using namespace std;
// Define a struct named 'Person'
struct Person {
    // Member variables
    string name;
    int age;
```

```
    // Member function
    void printInfo() {
        cout << "Name: " << name
```

```
int main(){
    Person person();
    person.printInfo();
}
```

```
> g++ basics001.cpp
basics001.cpp:17:18: warning: empty parentheses interpreted as a function declaration [-Wvexing-parse]
    Person person();
                  ^~
basics001.cpp:17:18: note: replace parentheses with an initializer to declare a variable
    Person person();
                  ^~
                  = {}
basics001.cpp:18:5: error: base of member reference is a function; perhaps you meant to call it with no arguments?
    person.printInfo();
    ^~~~~~
    ()
```

Struct [Constructor].....

```
#include <iostream>
using namespace std;
// Define a struct named 'Person'
struct Person {
    // Member variables
    string name;
    int age;

    // Member function
    void printInfo() {
        cout << "Name: " << name
        << ", Age: " << age << endl;
    }
};
```

Let's do some dissection.....

```
int main(){
    Person person; //or, Person person = {};
    // person.printInfo();
    person.printInfo();
}
```

```
> ./a.out
Name: , Age: 18635760
```



Struct [Constructor].....

```
#include <iostream>
using namespace std;
// Define a struct named 'Person'
struct Person {
    // Member variables
    string name;
    int age;

    // Default
    Person(){};

    // Member function
    void printInfo() {
        cout << "Name: " << name
        << ", Age: " << age << endl;
    }
};
```

Not having a constructor is equivalent to having empty constructor like below...

Let's do some dissection.....

```
int main(){
    Person person; //or, Person person = {};
    // person.printInfo();
    person.printInfo();
}
```

```
> ./a.out
Name: , Age: 18635760
```

?

Struct [Constructor].....

```
#include <iostream>
using namespace std;
// Define a struct named 'Person'
struct Person {
    // Member variables
    string name;
    int age;
    // Constructor
    Person(const string& n, int a) : name(n), age(a) {}

    // Member function
    void printInfo() {
        cout << "Name: " << name
        << ", Age: " << age << endl;
    }
};
```

What did we learn?

Constructor is a way of saying, this is how I want to define my structure...

```
18 int main(){
19     Person person("Keshav", 999);
20     person.printInfo();
21 }
```

> ./a.out

Name: Keshav, Age: 999

Struct [Constructor].....

```
struct Person {  
    // Member variables  
    string name;  
    int age;  
    // Option01. Default  
    Person(){};  
  
    // Option02. Initialized with name only,  
    // (here age is going to be garbage)  
    Person(const string& n):name(n){};  
  
    // Option03. You can initialize and do stuff  
    Person(const int& i):age(i){  
        name = "Name not provided";  
    };  
};
```

Note: We already did
Option2, that is
initializing with all
parameters

```
int main(){  
    // Option0  
    Person person;  
    person.printInfo();  
}
```

Name: , Age: 87497712

```
int main(){  
    // Option1  
    Person person("Keshav");  
    person.printInfo();  
}
```

> ./a.out
Name: Keshav, Age: 39836656

```
int main(){  
    // Option3  
    Person person(999);  
    person.printInfo();  
}
```

> ./a.out
Name: Name not provided, Age: 999

Struct [Constructor].....

```
struct Person {  
    // Member variables  
    string name;  
    int age;  
    // Option01. Default  
    Person(){};  
  
    // Option02. Initialized with name only,  
    // (here age is going to be garbage)  
    Person(const string& n):name(n){};  
  
    // Option03. You can initialize and do stuff  
    Person(const int& i):age(i){  
        name = "Name not provided";  
    };  
};
```

Note: We already did
Option2, that is
initializing with all
parameters

```
int main(){  
    // Option0  
    Person person;  
    person.printInfo();  
}
```

Name: , Age: 87497712

```
int main(){  
    // Option1  
    Person person("Keshav");  
    person.printInfo();  
}
```

> ./a.out
Name: Keshav, Age: 39836656

```
int main(){  
    // Option3  
    Person person(999);  
    person.printInfo();  
}
```

> ./a.out
Name: Name not provided, Age: 999

Struct [Constructor].....

```
#include <iostream>
#include <vector>
using namespace std;

struct Scores{
    string team;
    Scores(string const& t):team(t) {};
};
```

What is a best way to
reference the member?

```
#include <iostream>
#include <vector>
using namespace std;

struct Scores{
    string team;
    string town;
    Scores(string const& t, string const& t):team(t), town(t){};
};
```

Struct [Constructor].....

```
#include <iostream>
#include <vector>
using namespace std;

struct Scores{
    string team;
    Scores(string const& t):team(t) {};
};
```

What is a best way to
reference the member?

Resolution: do not use same
argument twice in
constructor, may be do t0, t1

```
#include <iostream>
#include <vector>
using namespace std;

struct Scores{
    string team;
    string town;
    Scores(string const& t0, string const& t1):team(t0), town(t1){};
};
```

Struct [Constructor].....

```
#include <iostream>
#include <vector>
using namespace std;

struct Scores{
    string team;
    Scores(string const& t):team(t) {};
};
```

What is a best way to reference the member?

Resolution: do not use same argument twice in constructor, may be do t0, t1

BUT THIS IS NOT A GOOD PRACTICE
WHAT IF...we can use actual variable name

```
#include <iostream>
#include <vector>
using namespace std;

struct Scores{
    string team;
    string town;
    Scores(string const& t0, string const& t1):team(t0), town(t1){};
};
```

Struct [Constructor].....

```
#include <iostream>
#include <vector>
using namespace std;

struct Scores{
    string team;
    Scores(string const& t):team(t) {};
};
```

What is a best way to reference the member?

Resolution: do not use same argument twice in constructor, may be do t0, t1

BUT THIS IS NOT A GOOD PRACTICE
WHAT IF...we can use actual variable name

```
#include <iostream>
#include <vector>
using namespace std;

struct Scores{
    string team;
    string town;
    Scores(string const& team, string const& town):team(team), town(town){};
};
```

Following operations are identical

```
#include <iostream>
#include <vector>
using namespace std;
```

```
struct Scores{
    string team;
    string town;
    Scores(string const& team, string const& town):team(team), town(town){};
};
```

```
struct Scores{
    string team;
    string town;
    Scores(string const& team, string const& town){
        this->team = team;
        this->town = town;
    };
};
```

Struct [Constructor].....

```
struct Scores{  
    string team;  
    string town;  
    Scores(string const& team, string const& town){  
        this->team = team;  
        this->town = town;  
    };  
};
```

'this' is called a pointer, which is a **reference** to the current instance of the structure
(Good Habit: Use this-> to resolve naming conflicts)

Struct [Constructor].....

```
struct Scores{  
    string team;  
    string town;  
    Scores(string const& team, string const& town){  
        this->team = team;  
        this->town = town;  
    };  
};
```

'this' is a pointer, which is a **reference** to the current instance of the structure
(Good Habit: Use this-> to resolve naming conflicts)

Struct [Constructor].....

```
struct Person{  
    string name;  
    Person(string const& name):name(name){};  
    void setName(string const& name){  
        // we have two conflicting variable,  
        // 1. the argument in setName function which is an input to our function  
        // 2. the original name of the person define in line 6 and initialized at line 7  
        // THIS IS A CONFLICT, now update the name using following approach:  
        this->name = name;  
    }  
};
```

‘this’ is a pointer, which is a **reference** to the current instance of the structure
(Good Habit: Use this-> to resolve naming conflicts)

Struct

CASE STUDY

Struct [...]......

```
#include <iostream>
#include <vector>
using namespace std;

struct Person{
    string name, ssn, pin, phone;
    Person(string na, string ss, string pi, string ph):
        name(na), ssn(ss), pin(pi), phone(ph) {}
};
```

Struct [...]......

```
#include <iostream>
#include <vector>
using namespace std;

struct Person{
    string name, ssn, pin, phone;
    Person(string na, string ss, string pi, string ph):
        name(na), ssn(ss), pin(pi), phone(ph) {}
};
```

```
int main(){
    // You logged into CreditCard Website and added your credential
    // As of server is running in background
    // Following is you signing up
    Person person("Me", "123-45-6789", "1234", "1234567890");
}
```

Struct [...]......

```
#include <iostream>
#include <vector>
using namespace std;

struct Person{
    string name, ssn, pin, phone;
    Person(string na, string ss, string pi, string ph):
        name(na), ssn(ss), pin(pi), phone(ph) {}
};
```

```
int main(){
    // You logged into CreditCard Website and added your credential
    // As of server is running in background
    // Following is you signing up
    Person person("Me", "123-45-6789", "1234", "1234567890");
}
```

Struct [...]......

Find the typo!!!

```
#include <iostream>
#include <vector>
using namespace std;

struct Person{
    string name, ssn, pin, phone;
    Person(string na, string ss, string pi, string ph):
        name(na), ssn(ss), pin(pi), phone(ph) {}
};
```

```
int main(){
    // You logged into CreditCard Website and added your credential
    // As of server is running in background
    // Following is you signing up
    Person person("Me", "123-45-6789", "1234", "1234567890");

    // Assume HACKER, has control after sometime and following is the
    // hacker or someone who is not you
    // Question... Can they do following?
    // Question... read your credentials..
    cout << person.name << endl; // yes they can
    cout << person.ssn << endl; // yes they can and so on...
    // Can they temper your data
    person.ssn = "xxx-yy-zzzz"; // yes they can
}
```

Struct [...]......

```
struct Person{
    string name, ssn, pin, phone;
    Person(string na, string ss, string pi, string ph):
        name(na), ssn(ss), pin(pi), phone(ph) {}
    void showData(){
        cout << " Name: " << this->name << endl;
        cout << " SSN: " << this->ssn << endl;
        cout << " PIN: " << this->pin << endl;
        cout << "Phone: " << this->phone << endl;
    }
};

int main(){
```

```
    // You logged into CreditCard Website and added your credential
    // As of server is running in background
    // Following is you signing up
    Person person("Me", "123-45-6789", "1234", "1234567890");
    // Assume HACKER, has control after sometime and following is the
    // hacker or someone who is not you
    // Question... Can they do following?
    // Question... read your credentials..
    cout << "Accessing credentials..." << endl;
    cout << person.name << endl; // yes they can
    cout << person.ssn << endl; // yes they can and so on...
    // Can they temper your data
    cout << "Printing all credentials..." << endl;
    person.ssn = "xxx-yy-zzzz"; // yes they can
    // Printing all credentials
    person.showData();
}
```

```
> ./a.out
Accessing credentials...
Me
123-45-6789
Printing all credentials...
  Name: Me
   SSN: xxx-yy-zzzz
   PIN: 1234
  Phone: 1234567890
```

Struct [ACCESS IDENTIFIER].....

LIMITING ACCESS....?

Struct [ACCESS IDENTIFIER].....

‘access-specifiers’ like, **private**, **protected**, **public** can be used to limit the access of member outside of struct, by default everything is public. As of now lets ignore **protected**.

Struct [ACCESS IDENTIFIER].....

```
struct SomeStruct{
```

```
    private:
```

```
        // everything private below...
```

ACCESS WITHIN
STRUCT ONLY

```
    public:
```

```
        // everything public below...
```

ACCESS FROM
MOCHAS JAVA..
OR ANY COFFEE PLACE YOU
LIKE....

```
}
```

Struct [ACCESS IDENTIFIER].....

WARNING!!! THIS IS A BAD CODE

```
#include <iostream>
#include <vector>
using namespace std;

struct Person{
private:
    string name;
    string ssn;
    string pin;
    string phone;
    bool validate(){
        string pin;
        cout << "Enter pin: ";
        cin >> pin;
        return pin == this->pin;
    };
};
```



```
public:
    vector<string> hobbies;
    Person(string n, string s, string p):name(n), ssn(s), pin(p) {}

    string getSSN(){
        if(validate()) return this->ssn;
        else return "000-00-000";
    }

    string getPhone(){
        if(validate()) return this->ssn;
        else return "000-000-0000";
    }

    void updatePhone(string const& phone){
        if(validate()){
            this->phone = phone;
            cout << "Phone Updated" << endl;
        }else
            cout << "Error!! User not validated" << endl;
    }
}
```

Struct [.....].....

```
#include <iostream>
using namespace std;

struct GameCard {
    string name;
    int score;
    GameCard(string n, int s) : name(n), score(s) { } // BAD!!!
};

int main(int argc, char const *argv[])
{
    /* code */
    GameCard card = GameCard("Resident Evil-4, Village", 1100);
    return 0;
}
```

What will when program ends?

Struct [Destructor].....

```
struct GameCard {  
    string name;  
    int score;  
    GameCard(string n, int s) : name(n), score(s) {  
        cout << "Game Card Created" << endl;  
    } // BAD!!!  
    ~GameCard() {  
        cout << "Game Card Deleted" << endl;  
    }  
};  
  
int main(int argc, char const *argv[])  
{  
    /* code */  
    GameCard card = GameCard("Resident Evil-4, Village", 1100);  
    return 0;  
}
```

```
> ./a.out  
Game Card Created  
Game Card Deleted
```

Struct [POINTERS].....

```
struct GameCard {
    string name;
    int score;
    GameCard(string n, int s) : name(n), score(s) {
        cout << "Game Card Created" << endl;
    } // BAD!!!
    ~GameCard() {
        cout << "Game Card Deleted" << endl;
    }
};

int main(int argc, char const *argv[])
{
    /* GameCard card = GameCard("Resident Evil-4, Village", 1100); */
    GameCard* pcard = new GameCard("Resident Evil-4, Village", 1100);
    return 0;
}
```



Struct [POINTERS].....

```
struct GameCard {
    string name;
    int score;
    GameCard(string n, int s) : name(n), score(s) {
        cout << "Game Card Created" << endl;
    } // BAD!!!
    ~GameCard() {
        cout << "Game Card Deleted" << endl;
    }
};

int main(int argc, char const *argv[])
{
    /* GameCard card = GameCard("Resident Evil-4, Village", 1100); */
    GameCard* pcard = new GameCard("Resident Evil-4, Village", 1100);
    delete pcard;
    return 0;
}
```



```
> ./a.out
Game Card Created
Game Card Deleted
```

You must de-allocate memory while using pointers.

... [....]

```
#include <iostream>
using namespace std;
struct GameCard {
    string name;
    int score;
    GameCard(string n, int s) : name(n), score(s) {}
    void printInfo(){
        cout << "Game: " << this->name << endl;
        cout << "Score: " << this->score << endl;
    }
};

int main()
{
    GameCard card = GameCard("Resident Evil-4, Village", 1100);
    card.printInfo();
    return 0;
}
```

```
> ./a.out
```

```
Game: Resident Evil-4, Village
Score: 1100
```

... [...]....

```
#include <iostream>
using namespace std;
struct GameCard {
    string name;
    int score;
    GameCard(string n, int s) : name(n), score(s) {}
    void printInfo(){
        cout << "Game: " << this->name << endl;
        cout << "Score: " << this->score << endl;
    }
};

int main()
{
    GameCard card = GameCard("Resident Evil-4, Village", 1100);
    card.printInfo();
    return 0;
}
```

```
> ./a.out
```

```
Game: Resident Evil-4, Village
Score: 1100
```

... [...]....

```
int main()
{
    GameCard card = GameCard("Resident Evil-4, Village", 1100);
    cout << card;
    return 0;
}
```

Can we do this?

Expectation !!

```
> ./a.out
Game: Resident Evil-4, Village
Score: 1100
```

YES.. Operator OVerloading..

... [...] ...

YES

OPERATOR OVERLOADING

Unit-04.01

OP.OVER-LOADING [...]......

```
using namespace std;
struct GameCard {
    string name;
    int score;
    GameCard(string n, int s) : name(n), score(s) {}
};

void operator<<(ostream& COUT, GameCard& card){
    COUT << "Game: " << card.name << endl;
    COUT << "Score: " << card.score << endl;
}

int main()
{
    GameCard card = GameCard("Resident Evil-4, Village", 1100);
    cout << card;
    return 0;
}
```

```
> ./a.out
```

```
Game: Resident Evil-4, Village
Score: 1100
```

OP.OVER-LOADING [...]......

```
using namespace std;
struct GameCard {
    string name;
    int score;
    GameCard(string n, int s) : name(n), score(s) {}
};

void operator<<(ostream& COUT, GameCard& card){
    COUT << "Game: " << card.name << endl;
    COUT << "Score: " << card.score << endl;
}

int main()
{
    GameCard card = GameCard("Resident Evil-4, Village", 1100);
    cout << card << endl;
    return 0;
}
```

This will fail.. But why?

OP.OVER-LOADING [...]......

```
#include <iostream>
using namespace std;
struct GameCard {
    string name;
    int score;
    GameCard(string n, int s) : name(n), score(s) {}
};

ostream& operator<<(ostream& COUT, GameCard& card){
    COUT << "Game: " << card.name << endl;
    COUT << "Score: " << card.score << endl;
    return COUT;
}

int main()
{
    GameCard card = GameCard("Resident Evil-4, Village", 1100);
    cout << card << endl;
    return 0;
}
```

```
> ./a.out
Game: Resident Evil-4, Village
Score: 1100
```

Previously...

```
> ./a.out
Game: Resident Evil-4, Village
Score: 1100
```

We can do interesting things like..

OP.OVER-LOADING [...]......

```
#include <iostream>
#include <map>
#include <iomanip>

using namespace std;
struct GameCard {
    map<string, int> games;
    int score = 0;
    GameCard(map<string, int> games) : games(games){
        for(const auto& [game, score] : games){
            this->score += score;
        }
    };
};
```

```
ostream& operator<<(ostream& COUT, GameCard& card){
    cout << setw(25) << "Games"
         << setw(10) << "Score"
         << endl;
    for(const auto& [game, score] : card.games){
        cout << setw(25) << game
             << setw(10) << score
             << endl;
    }
    return COUT;
}
```

```
int main()
{
    GameCard card = GameCard(
        {
            {"Resident Evil-4, Village", 1100},
            {"Assasin Creed, Mirage", 800},
            {"Assasin Creed, Valhalla", 1300},
            {"GTA-V", 2500},
        }
    );
    cout << card << endl;
    return 0;
}
```

```
> g++ -std=c++17 overloading_1.cpp
> ./a.out
```

Games	Score
Assasin Creed, Mirage	800
Assasin Creed, Valhalla	1300
GTA-V	2500
Resident Evil-4, Village	1100

OP.OVER-LOADING [...]......

```
int main()
{
    GameCard card0 = GameCard(
        {
            {"Resident Evil-4, Village", 1100},
            {"Assasin Creed, Mirage", 800},
            {"Assasin Creed, Valhalla", 1300},
            {"GTA-V", 2500},
        }
    );
    GameCard card1 = GameCard(
        {
            {"Tom Clancy's, Ghost Recon", 100},
            {"Sniper Elite 5", 3800},
        }
    );

    GameCard card = card0 + card1;
    cout << card << endl;
    return 0;
}
```

Can you modify
previous program to achieve this?

```
> g++ -std=c++17 overloading_1.cpp
> ./a.out
```

Games	Score
Assasin Creed, Mirage	800
Assasin Creed, Valhalla	1300
GTA-V	2500
Resident Evil-4, Village	1100
Sniper Elite 5	3800
Tom Clancy's, Ghost Recon	100

OP.OVER-LOADING [...]......

```
}  
? ? ( ? ? ? ? ) {  
    ? ? ? ;  
    for( ? : ? ) {  
        ?  
        ?  
    }  
    for( ? : ? ) {  
        ?  
        ?  
    }  
    ?  
}
```

```
int main()  
{  
    GameCard card0 = GameCard(  
        {"Resident Evil-4, Village", 1100},  
        {"Assasin Creed, Mirage", 800},  
        {"Assasin Creed, Valhalla", 1300},  
        {"GTA-V", 2500},  
    );  
    GameCard card1 = GameCard(  
        {"Tom Clancy's, Ghost Recon", 100},  
        {"Sniper Elite 5", 3800},  
    );  
  
    GameCard card = card0 + card1;  
    cout << card << endl;  
    return 0;  
}
```

```
> g++ -std=c++17 overloading_1.cpp  
> ./a.out
```

Games	Score
Assasin Creed, Mirage	800
Assasin Creed, Valhalla	1300
GTA-V	2500
Resident Evil-4, Village	1100
Sniper Elite 5	3800
Tom Clancy's, Ghost Recon	100

OP.OVER-LOADING [...].....

```
GameCard operator+(GameCard& card0, GameCard& card1){
    GameCard card = GameCard({});
    for(const auto& [game, score] : card0.games){
        card.games[game] = score;
        card.score += score;
    }
    for(const auto& [game, score] : card1.games){
        card.games[game] = score;
        card.score += score;
    }
    return card;
}
```

```
int main()
{
    GameCard card0 = GameCard(
    {
        {"Resident Evil-4, Village", 1100},
        {"Assasin Creed, Mirage", 800},
        {"Assasin Creed, Valhalla", 1300},
        {"GTA-V", 2500},
    }
    );
    GameCard card1 = GameCard(
    {
        {"Tom Clancy's, Ghost Recon", 100},
        {"Sniper Elite 5", 3800},
    }
    );

    GameCard card = card0 + card1;
    cout << card << endl;
    return 0;
}
```

```
> g++ -std=c++17 overloading_1.cpp
> ./a.out
```

Games	Score
Assasin Creed, Mirage	800
Assasin Creed, Valhalla	1300
GTA-V	2500
Resident Evil-4, Village	1100
Sniper Elite 5	3800
Tom Clancy's, Ghost Recon	100

Unary Operators:

1. `+` : Unary plus
2. `-` : Unary minus
3. `++` : Pre-increment and Post-increment
4. `--` : Pre-decrement and Post-decrement
5. `*` : Dereference (used with pointers)
6. `&` : Address-of
7. `!` : Logical NOT
8. `~` : Bitwise NOT (Complement)

Others:

1. `[]` : Subscripting (array access)
2. `()` : Function call
3. `->` : Member access through pointer
4. `->*` : Pointer to member access
5. `new` : Dynamic memory allocation
6. `delete` : Dynamic memory deallocation

I GAVE YOU THE RECIEPE
NOW BE THE CHEF...
...check if all of these are valid?

FIGURE OUT WHAT IS NOT POSSIBLE

Assignment Operators:

1. `=` : Simple assignment
2. `+=` : Addition assignment
3. `-=` : Subtraction assignment
4. `*=` : Multiplication assignment
5. `/=` : Division assignment
6. `%=` : Modulus assignment
7. `&=` : Bitwise AND assignment
8. `|=` : Bitwise OR assignment
9. `^=` : Bitwise XOR assignment
10. `<<=` : Left shift assignment
11. `>>=` : Right shift assignment

Binary Operators:

1. `+` : Addition
2. `-` : Subtraction
3. `*` : Multiplication
4. `/` : Division
5. `%` : Modulus
6. `==` : Equality
7. `!=` : Inequality
8. `<` : Less than
9. `>` : Greater than
10. `<=` : Less than or equal to
11. `>=` : Greater than or equal to
12. `&&` : Logical AND
13. `||` : Logical OR
14. `&` : Bitwise AND
15. `|` : Bitwise OR
16. `^` : Bitwise XOR
17. `<<` : Left shift
18. `>>` : Right shift
19. `,` : Comma

CARD: card0

Games	Score
Assasin Creed, Mirage	800
Assasin Creed, Valhalla	1300
GTA-V	2500
Resident Evil-4, Village	1100
Total Score	5700

CARD: card1

Games	Score
Sniper Elite 5	3800
Tom Clancy's, Ghost Recon	100
Total Score	3900

CARD: card2

Games	Score
Limbo	1000
Total Score	1000

CARD: card = card0+card1

Games	Score
Assasin Creed, Mirage	800
Assasin Creed, Valhalla	1300
GTA-V	2500
Resident Evil-4, Village	1100
Sniper Elite 5	3800
Tom Clancy's, Ghost Recon	100
Total Score	9600

CARD: card += card2

Games	Score
Assasin Creed, Mirage	800
Assasin Creed, Valhalla	1300
GTA-V	2500
Limbo	1000
Resident Evil-4, Village	1100
Sniper Elite 5	3800
Tom Clancy's, Ghost Recon	100
Total Score	10600

CARD: card -= card0

Games	Score
Limbo	1000
Sniper Elite 5	3800
Tom Clancy's, Ghost Recon	100
Total Score	4900

Can you extend our program?